MODULE 1

Big Picture & Safety Primer

Duration	Format	Audience	Day / Position
45–60 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Opening**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Explain what vibe coding is, where the term originated, and how it fits into professional software development at TI
2. Describe how AI (specifically Claude Code) integrates into each phase of the SDLC
3. Identify the strengths and limitations of AI-assisted coding for both IT application work and process engineering tasks
4. Apply a practical Privacy/IP/Licensing checklist before using AI tools on any TI codebase
5. Successfully install, authenticate, and run their first Claude Code command in a course repository

SESSION TIMING BREAKDOWN

0:00–0:12	**Section 1: What Is Vibe Coding? — Origin, Definition & Spectrum**
0:12–0:25	**Section 2: AI in the SDLC — Where Claude Code Fits**
0:25–0:35	**Section 3: Strengths vs. Limits — Honest Assessment**
0:35–0:45	**Section 4: Privacy / IP / Licensing Checklist for TI**
0:45–0:55	**Section 5: Hands-On Setup — Claude Code Installation & First Run**
0:55–1:00	**Wrap-Up: Course Repo Tour & Guardrails**

0:00 – 0:12	SECTION 1: What Is Vibe Coding?

1. What Is Vibe Coding?

1.1 The Origin Story

The term **vibe coding** was coined by **Andrej Karpathy** (co-founder of OpenAI, former Director of AI at Tesla) on February 6, 2025. In a post on X (formerly Twitter) that went viral with millions of views, he wrote:

> *"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works."*
>
> — Andrej Karpathy, February 2025

Karpathy described using AI tools (specifically Cursor with Claude Sonnet) combined with voice input via SuperWhisper, barely touching the keyboard. He would accept all AI suggestions without reviewing diffs, paste error messages back to the AI without comment, and let the codebase grow beyond his own comprehension. He deliberately called this approach suitable for "throwaway weekend projects."

1.2 The Vibe Coding Spectrum

In practice, "vibe coding" has evolved to mean different things to different people. Simon Willison, a well-known Python developer, drew an important distinction:

> *"If an LLM wrote every line of your code, but you've reviewed, tested, and understood it all, that's not vibe coding in my book—that's using an LLM as a typing assistant."*

This gives us a practical spectrum that your team at TI needs to understand:

Level	Approach	Appropriate For
Pure Vibe Coding	Accept all AI output without review. Trust the vibes.	Weekend prototypes, throwaway experiments, learning explorations
Guided Vibe Coding	Use AI for first drafts, review key logic, test outputs, iterate via prompts.	Internal tools, proof-of-concepts, rapid automation scripts
AI-Assisted Engineering	AI generates code; engineer reviews, tests, understands, and owns every line.	Production systems, TI applications, process-critical code

☆ **KEY POINT:** This course teaches AI-Assisted Engineering (Level 3) using Claude Code. You will leverage AI for speed and productivity, but you will always review, test, and understand the code. This is the professional standard for enterprise environments like TI.

1.3 Why This Matters Now

- Y Combinator reported that 25% of startups in their Winter 2025 batch had codebases that were 95% AI-generated
- In July 2025, The Wall Street Journal reported that professional engineers were adopting AI coding for commercial use cases
- Anthropic released Claude Code v2.0 in September 2025 with VS Code extension, checkpoints, and subagents
- By early 2026, major enterprises (including semiconductor companies) are integrating AI coding into daily workflows

0:12 – 0:25	SECTION 2: AI in the SDLC

2. AI in the Software Development Lifecycle

AI-assisted coding isn't just about writing code faster. Claude Code can assist at every phase of the SDLC. Here's how it maps to your daily work at TI:

SDLC Phase	Claude Code Capability	TI Use Case Example
Requirements	Analyze docs, generate user stories, clarify specs via conversation	Parse a process spec and generate Python task requirements
Design	Propose architecture, create design docs, suggest patterns	Design a data pipeline architecture for fab sensor data
Implementation	Generate code, scaffold projects, refactor, add types/docs	Scaffold a Python ETL script for test data processing
Testing	Generate unit/integration tests, fixtures, mocks, coverage analysis	Auto-generate pytest suites for a yield analysis module
Code Review	Review PRs, identify bugs, suggest improvements, check standards	Review a colleague's Python script for edge cases and type safety
Deployment	CI/CD setup, GitHub Actions, release notes, changelog generation	Set up automated lint/test pipeline for an internal tool
Maintenance	Debug from tracebacks, explain legacy code, refactor for clarity	Understand and modernize a 3-year-old batch processing script
Documentation	Generate README, API docs, inline comments, architectural docs	Auto-document a process automation library for team handoff

2.1 What Is Claude Code, Specifically?

Claude Code is Anthropic's agentic coding tool. Unlike simple autocomplete or chat-based assistants, Claude Code is an autonomous agent that:

Capability	What This Means In Practice
Lives in your terminal	You invoke it with the claude command, right where you already work
Understands your codebase	It reads your files, understands project structure, and maintains context
Takes real actions	It can create files, edit code, run commands, execute tests — not just suggest
Manages git workflows	It creates branches, commits, generates PR descriptions, resolves conflicts

Has a VS Code extension	See real-time diffs in your IDE sidebar while Claude works
Supports subagents	Delegate specialized tasks to code reviewer, debugger, or planner agents
Includes safety features	Checkpoints to rewind, permission model, dry-run mode, allow-lists

2.2 Claude Code vs. Other AI Coding Tools

For context, here's how Claude Code compares to tools you may have heard about. This course exclusively uses Claude Code:

Feature	Claude Code	Other Tools (Copilot, Codex, etc.)
Interface	Terminal CLI + VS Code extension	IDE plugins, web interfaces
Agency	Fully agentic — reads, writes, executes, tests	Mostly suggestion-based or sandboxed
Context	Entire codebase + @-file references	Usually limited to open file/tab
Safety	Checkpoints, rewind, permission model, hooks	Varies, often less granular
Customization	CLAUDE.md, slash commands, subagents, hooks	Limited configuration
CI/CD Integration	Headless mode (claude -p) for pipelines	Some support, often separate
Pricing	Claude Pro/Max subscription or API pay-per-use	Varies by tool/provider

☆ **KEY POINT:** We chose Claude Code for this course because it offers the deepest agentic capabilities: it doesn't just suggest code, it reads your project, takes actions, runs your tests, and manages your git workflow. This makes it the most comprehensive tool for learning professional AI-assisted engineering.

0:25 – 0:35	SECTION 3: Strengths vs. Limits

3. Strengths vs. Limitations — An Honest Assessment

Before you start using Claude Code, it's critical to understand what it does well and where it falls short. Being realistic about capabilities will save you hours of frustration and prevent costly mistakes.

3.1 What Claude Code Does Well

Strength	Practical Impact
Boilerplate elimination	Scaffolds projects, config files, Docker setups, CI pipelines in seconds instead of hours
Test generation	Generates comprehensive test suites including edge cases you might miss, with fixtures and mocks
Code comprehension	Explains unfamiliar codebases, traces execution paths, identifies patterns across large repos
Refactoring at scale	Renames across files, extracts functions, adds type hints, updates imports consistently
Documentation	Generates docstrings, README files, API docs, inline comments from existing code
Debugging speed	Paste a traceback, get a diagnosis and fix. Often resolves issues in one prompt
Language translation	Converts between languages (e.g., MATLAB to Python) while preserving logic
Pattern application	Applies consistent coding patterns across an entire codebase

3.2 Where Claude Code Has Limitations

Limitation	What This Means For You
Hallucination	May confidently generate code that uses non-existent APIs, wrong function signatures, or fabricated libraries. Always verify.
No real-time data	Cannot access live systems, databases, or APIs unless explicitly connected via MCP servers
Context window limits	Very large codebases may exceed context. Use /compact and @-references to manage what Claude sees
Domain-specific knowledge	Limited knowledge of proprietary TI frameworks, internal APIs, or semiconductor-specific algorithms
Subtle logic bugs	Can produce code that looks correct, passes basic tests, but has edge-case logic errors. Critical code needs thorough review.

Security blind spots	May generate code with SQL injection, hardcoded secrets, or insecure defaults if not prompted carefully
Non-deterministic output	Same prompt may produce different code each time. Results are probabilistic, not guaranteed.
Over-engineering	Sometimes produces unnecessarily complex solutions when a simpler approach would suffice

⚠ **WARNING:** Never blindly trust AI-generated code in a production environment. The Lovable incident (May 2025) showed that 170 out of 1,645 AI-built web apps had security vulnerabilities exposing personal information. Professional AI-assisted engineering means: generate fast, review thoroughly, test rigorously.

3.3 The Right Mental Model

Think of Claude Code as a brilliant, extremely fast junior engineer who:

- Has read millions of codebases and knows patterns from all of them
- Works at superhuman speed but sometimes makes confident mistakes
- Never gets tired, never gets frustrated, and can iterate 50 times without complaint
- Needs clear direction and benefits enormously from specific, detailed prompts
- Does NOT understand your business logic, your TI-specific systems, or the real-world consequences of bugs

☆ **KEY POINT:** Your job is not to write code anymore. Your job is to think clearly, direct precisely, review critically, and test thoroughly. Claude Code handles the typing. You handle the thinking.

GROUP DISCUSSION (3 minutes)

- In pairs, discuss: What is one task in your daily work that you think AI could help with, and one task where you think human judgment is irreplaceable?
- IT engineers: think about your application development and deployment workflows
- Process engineers: think about your Python scripting and data analysis tasks
- Be ready to share one insight with the group

0:35 – 0:45	SECTION 4: Privacy / IP / Licensing Checklist

4. Privacy, IP & Licensing Checklist for TI

Before using any AI coding tool on company code, every engineer at TI needs to understand the risks and the guardrails. This section gives you a practical, actionable checklist you can apply from Day 1.

4.1 The Three Risk Categories

Risk Category	What Could Go Wrong	Mitigation
Data Exposure	Proprietary TI code, algorithms, or data gets sent to external AI APIs and could be stored, logged, or used for training	Use API with data retention controls; never paste secrets, credentials, or classified code into prompts
IP Contamination	AI generates code that is substantially similar to GPL or copyleft-licensed code in its training data, creating licensing obligations for TI	Always review generated code for attribution; use the PR checklist; avoid asking AI to reproduce specific open-source implementations
Code Quality Risk	AI generates code with security vulnerabilities, logic errors, or non-compliant patterns that make it into production	Mandatory code review for all AI-generated code; automated testing; never skip the review step

4.2 Anthropic's Data Handling (Claude Code)

Understanding how your data is handled when using Claude Code:

Aspect	Anthropic's Policy for Claude Code
API data retention	Anthropic does not train on API inputs/outputs by default. Data is retained for 30 days for abuse monitoring, then deleted.
Claude Pro/Max	Consumer-tier usage: conversations may be used for training unless you opt out. Use API for sensitive work.
Code execution	Claude Code runs commands locally on YOUR machine. Your code stays on your computer.
Context window	Code sent to Claude goes to Anthropic's API for inference, then the response comes back. The code is in transit.
Session logs	Conversation history is stored locally in ~/.claude/ on your machine.

Enterprise options	Anthropic offers enterprise plans; TI can also run Claude Code via Amazon Bedrock or Google Vertex for additional data controls.

4.3 The TI Safety Checklist

Use this checklist before every AI-assisted coding session. We'll reference it throughout the course:

✅ PRE-SESSION SAFETY CHECKLIST

- CLASSIFY your code: Is it public, internal, confidential, or restricted? Only use AI tools on code at the appropriate classification level per TI policy.
- STRIP SECRETS: Never include API keys, passwords, database connection strings, internal hostnames, or customer data in prompts.
- SCRUB PROPRIETARY ALGORITHMS: If your code contains patented or trade-secret algorithms, do not send those sections to the AI. Work on the surrounding code instead.
- CHECK YOUR PLAN: Are you using Claude Code via API (recommended for sensitive work) or via Claude Pro/Max subscription?
- REVIEW ALL OUTPUT: Every line of AI-generated code must be reviewed before committing. No exceptions.
- RUN TESTS: AI-generated code must pass your existing test suite plus new tests for the generated functionality.
- TAG AI CODE: Use a commit message convention (e.g., [ai-assisted]) so teammates know which code was AI-generated.
- CHECK LICENSES: If AI generates code that looks like it might come from an open-source project, verify the license compatibility.

4.4 What NOT to Send to Claude Code

⚠ **WARNING:** NEVER send the following to any AI tool, including Claude Code:

- Customer data, PII (personally identifiable information), or GDPR-protected information
- TI trade secrets, proprietary algorithms, or patented implementations
- Security credentials: API keys, SSH keys, passwords, tokens, certificates
- Internal infrastructure details: server names, IP addresses, network architecture
- Export-controlled technology or ITAR-regulated materials
- Compliance-sensitive code subject to regulatory audit

4.5 Safe Alternatives for Sensitive Code

When you need AI help with code that contains sensitive elements:

- Abstract the problem: Replace proprietary variable names with generic ones (sensor_data instead of ti_fab7_yield_data)

- Work on the pattern, not the content: Ask Claude Code to help with the algorithm structure, then manually adapt it to your proprietary context
- Use CLAUDE.md guardrails: Configure your project's CLAUDE.md to instruct Claude Code about what it should never touch or output
- Isolate sensitive sections: Keep proprietary code in separate files that you don't include in Claude Code's context

0:45 – 0:55	SECTION 5: Hands-On Setup — Installation & First Run

5. Hands-On: Claude Code Installation & First Run

5.1 Prerequisites Verification

Before installing Claude Code, verify these are already set up on your machine:

```
# Check Python version (need 3.10+)
python3 --version

# Check Git
git --version

# Check Node.js (needed for npm install method)
node --version    # Need 18+

# Check VS Code
code --version
```

5.2 Installing Claude Code

Claude Code offers two installation methods. The native installer is recommended (no Node.js required):

Method 1: Native Installer (Recommended)

```
# macOS / Linux
curl -fsSL https://claude.ai/install.sh | sh

# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex

# Verify installation
claude --version
```

Method 2: npm (Legacy, if native installer fails)

```
# Only if native installer doesn't work
npm install -g @anthropic-ai/claude-code
claude --version
```

Method 3: Homebrew (macOS) / winget (Windows)

```
# macOS
brew install claude-code

# Windows
winget install Anthropic.ClaudeCode
```

5.3 Authentication

Claude Code needs to connect to an Anthropic account. You have two options:

Method	Steps
OAuth (Pro/Max subscription)	Run claude in your terminal. Follow the one-time OAuth flow to connect your Claude Pro or Max account. This is simplest for training.
API Key	Get a key from console.anthropic.com. Set it: export ANTHROPIC_API_KEY=sk-ant-... Add to your shell profile (.bashrc, .zshrc) for persistence.

5.4 VS Code Extension Installation

```
# Option 1: From VS Code
# Open VS Code > Extensions (Ctrl+Shift+X)
# Search: 'Claude Code'
# Install the Anthropic official extension

# Option 2: From terminal
code --install-extension anthropic.claude-code
```

5.5 Your First Claude Code Command

Now let's run Claude Code for the first time. Navigate to the course repository:

```
# Clone the course repository
git clone https://github.com/datatweets/ti-vibe-coding.git
cd ti-vibe-coding

# Launch Claude Code
claude

# You should see the Claude Code interactive prompt.
# Type your first command:

> Explain the structure of this repository and list the main files
```

🚀 HANDS-ON EXERCISE: First Run (5 minutes)

- Clone the course repository and cd into it
- Run claude to start an interactive session
- Ask: "Explain the structure of this repository and list the main files"
- Ask: "What Python version does this project require?"
- Ask: "Show me the contents of the README"
- Type /help to see available slash commands
- Type exit or Ctrl+C to close the session
- Raise your hand if you get stuck at any step — that's what we're here for!

☆ **KEY POINT:** Congratulations! If you just ran those commands, you've already used AI to analyze a codebase faster than manually reading through files. This is the foundation everything else in this course builds on.

0:55 – 1:00	WRAP-UP: Course Repo Tour & Guardrails

6. Course Repository Tour & Guardrails

6.1 Repository Structure

The course repo is organized to support both audience tracks throughout the two days:

```
ti-vibe-coding
├── CLAUDE.md
├── labs
│   ├── lab1_scaffold
│   ├── lab2_assistant
│   └── lab3_sprint
├── README.md
└── src
    └── data
```

6.2 The CLAUDE.md File

This is the most important file for Claude Code. It's a project context file that tells Claude about your project, coding standards, and guardrails. Let's look at the course repo's CLAUDE.md:

```
# CLAUDE.md - Course Project Context

## Project
This is a Python training project for the TI Vibe Coding course.
Target: Python 3.10+ on Linux/macOS/Windows.

## Coding Standards
- Use Python type hints on all function signatures
- Follow PEP 8 via ruff and black formatters
- Write docstrings for all public functions (Google style)
- Maximum function length: 30 lines

## Testing
- Use pytest for all tests
- Minimum test coverage target: 80%
- Test files mirror source structure: src/core/calc.py ->
tests/unit/test_calc.py

## Safety Guardrails
- NEVER generate or reference real TI proprietary systems
- NEVER output API keys, credentials, or secrets
- ALWAYS use sample/synthetic data, not real production data
- When unsure about a change, explain the plan before executing
```

> ☆ **KEY POINT:** CLAUDE.md is like an onboarding document for your AI teammate. The more specific and well-written it is, the better Claude Code performs. We'll create and refine CLAUDE.md files throughout this course.

6.3 Course Guardrails

To keep this training productive and safe, we follow these ground rules:

COURSE GROUND RULES

- DO use the course repository and sample data for all exercises
- DO experiment freely — Claude Code checkpoints let you rewind any mistake
- DO ask questions at any time — there are no dumb questions in AI-assisted development
- DO NOT use real TI production code, customer data, or proprietary algorithms during training
- DO NOT share your API key or authentication credentials with other participants
- DO NOT run destructive commands (rm -rf, database drops) without instructor guidance
- REPORT any unexpected behavior, errors, or concerning AI outputs to the instructor

---

MODULE 1 RECAP

✅	Vibe coding was coined by Karpathy (Feb 2025). Professional use = AI-Assisted Engineering where you review everything.
✅	Claude Code is an agentic tool: it reads, writes, executes, and manages your code — not just suggests.
✅	AI is strong at boilerplate, tests, refactoring, docs. Weak at domain logic, security, and novel algorithms.
✅	Always apply the safety checklist: classify code, strip secrets, review output, run tests, tag AI commits.
✅	Claude Code is installed, authenticated, and you've run your first codebase analysis command.
✅	The course repo has CLAUDE.md guardrails, lab files, and pre-configured subagents ready for the next modules.

TRANSITION TO MODULE 2

Now that you understand the big picture and have Claude Code running, we're ready to learn the language that makes AI-assisted coding effective: prompting. In Module 2, you'll master the techniques that turn vague requests into precise, high-quality code generation.